



RAPPORT

Traitement d'image



université
PARIS-SACLAY

IUT DE CACHAN

17 JANVIER 2023

ROBINSAN
Kiritheepan

Table des matières

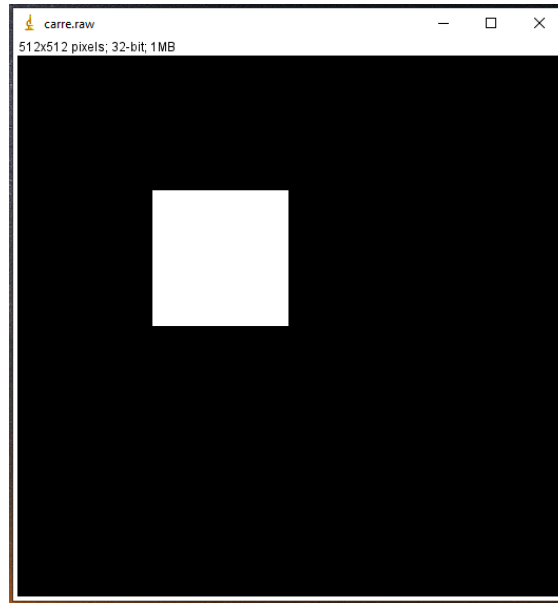
1	Traitement d'image :.....	2
1.1	Détection de contours : filtre 2D (ImageJ).....	2
1.1.1	Images sans bruit :	2
1.1.2	Images avec ajout de bruit gaussien :.....	6
1.1.3	Images avec ajout de bruit poivre et sel :	10
2	Estimation de performances sur cibles embarquées :.....	15
2.1	On remarque que la complexité arithmétique d'un filtre 2D de taille Nnoyau est :	15
2.2	La complexité arithmétique du calcul du gradient avec un filtre de Sobel :	16
2.3	Calcule de puissance de calcul en GFLOPS et les FPS :	16
3	Matlab :	17
	Effectuez sous Matlab, la détection de contours des images :	17
3.1	Réduction du temps d'exécution sous Matlab (Ferrari rouge et Ferrari jaune) :	19
4	Traitement d'un flux vidéo (programmation C/C++) :	20
5	Conclusion :	20

1 Traitement d'image :

1.1 Détection de contours : filtre 2D (ImageJ)

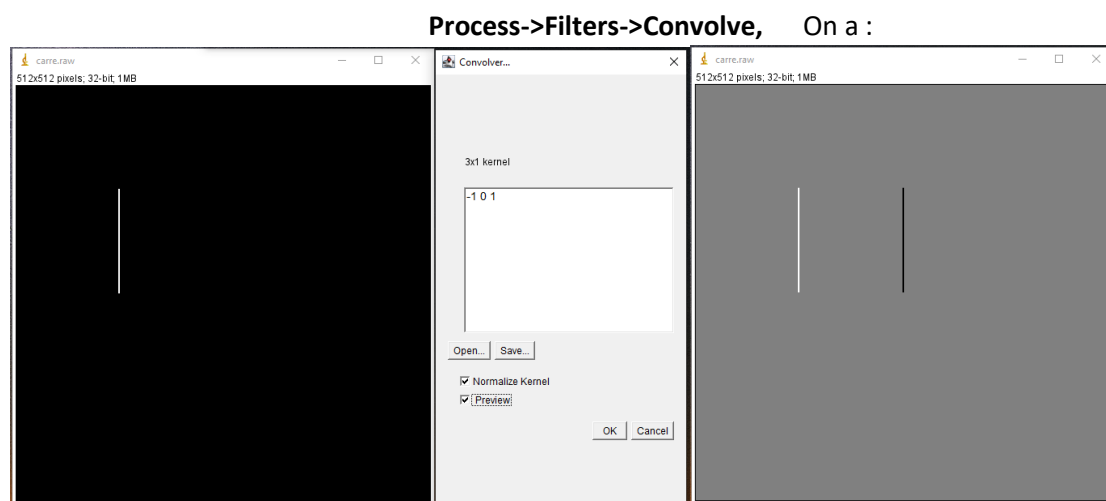
J'ai d'abord effectué la détection des bords à l'aide d'ImageJ, Logiciel de traitement d'image facile à utiliser. Je travaillerai avec deux Image : Un carré blanc et une Ferrari. Cette détection se fera avec ou sans bruit.

1.1.1 Images sans bruit :



J'ai ouvert l'image carre.raw (512*512 float 32 bits), le format est brut. Les données de cette image sont stockées en format binaire. J'ai effectué les actions suivantes : **File->Import->Raw**, Ainsi j'ai dû indiquer la hauteur et largeur, le type **32 bit real** et cochez la case **little endian** pour obtenir ce résultat sur ImageJ.

Afin de faire sortir les contours verticaux, j'ai utilisé le concepts $\text{abs}(D_x)$, c'est-à-dire, dérivée selon x ce qui correspond à effectuer les commandes suivantes sur ImageJ :



Puis, j'ai ajouté la valeur absolue via la commande **Process->Maths->abs**, on a :

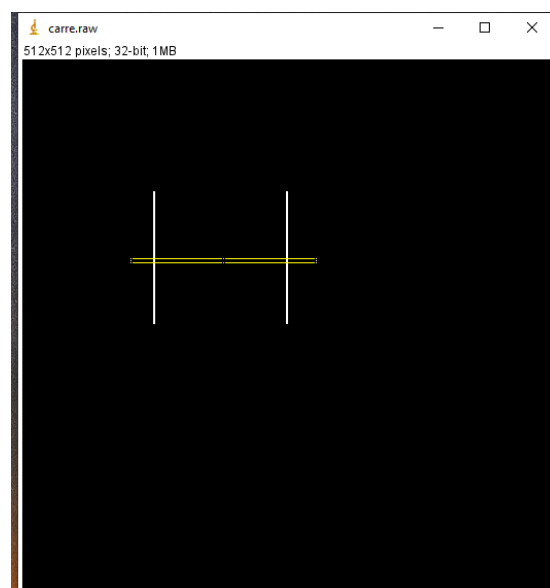


Calcul du dérivé permet avoir le contour, Variation **brutale entre pixels voisins** $\Rightarrow$ calcul de **la dérivée discrète**. On remarque, ici avec une **dérivée sur x en valeur absolue**, on a eu **le contour vertical** de l'image carrée du départ.

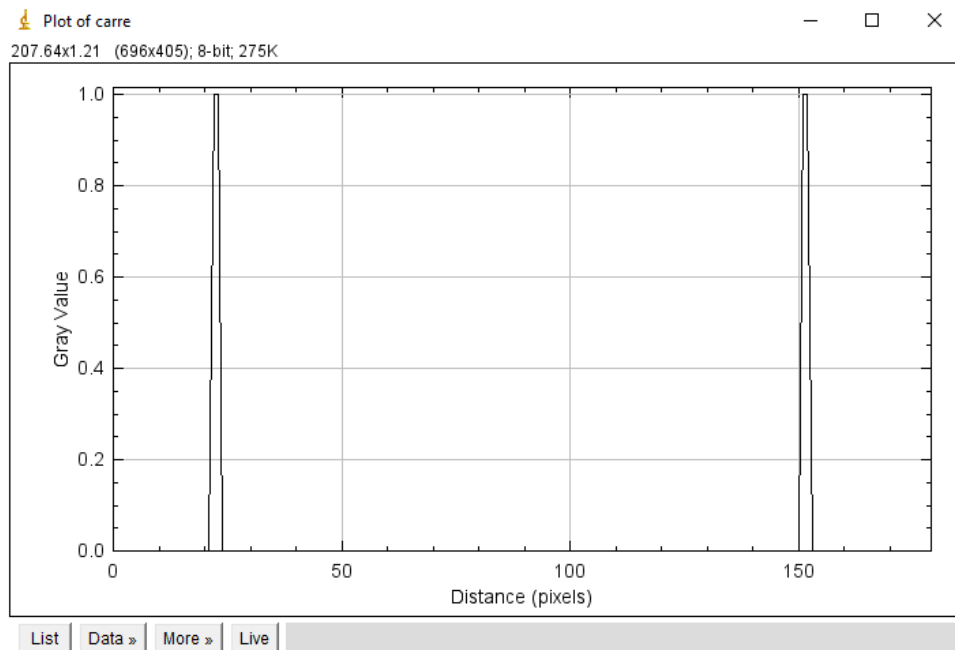
Puis j'essayais d'observer un profil (ou coupe) de cette image grac :

Analyse->Plot Profile

Sélection d'une ligne horizontale de pixels :



On a :

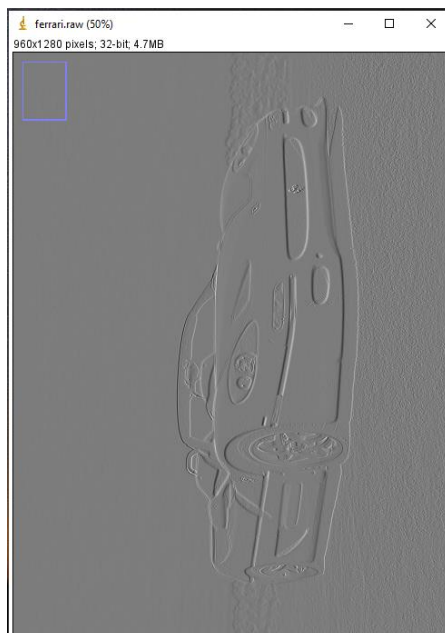


On remarque que, la résolution des valeurs de gris est à 1 vers 25 et 150 distances (pixels), cela montre, la fonction de l'opération $\text{abs}(Dx)$ graphiquement.

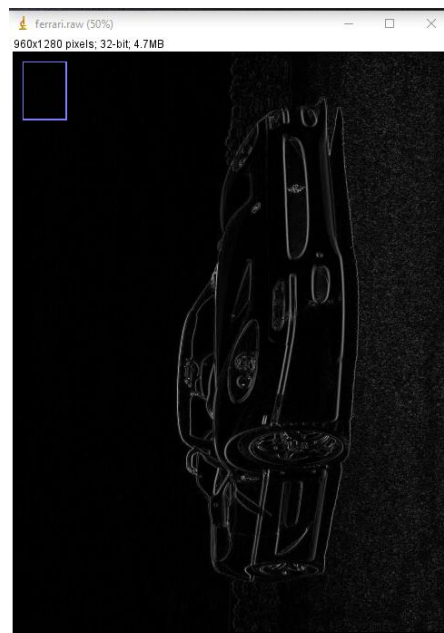
J'ai réalisé les mêmes études que sur `carre.raw` sur l'image `ferrari_NB.raw`, j'ai obtenu les résultats suivants :



Dérivée selon x :



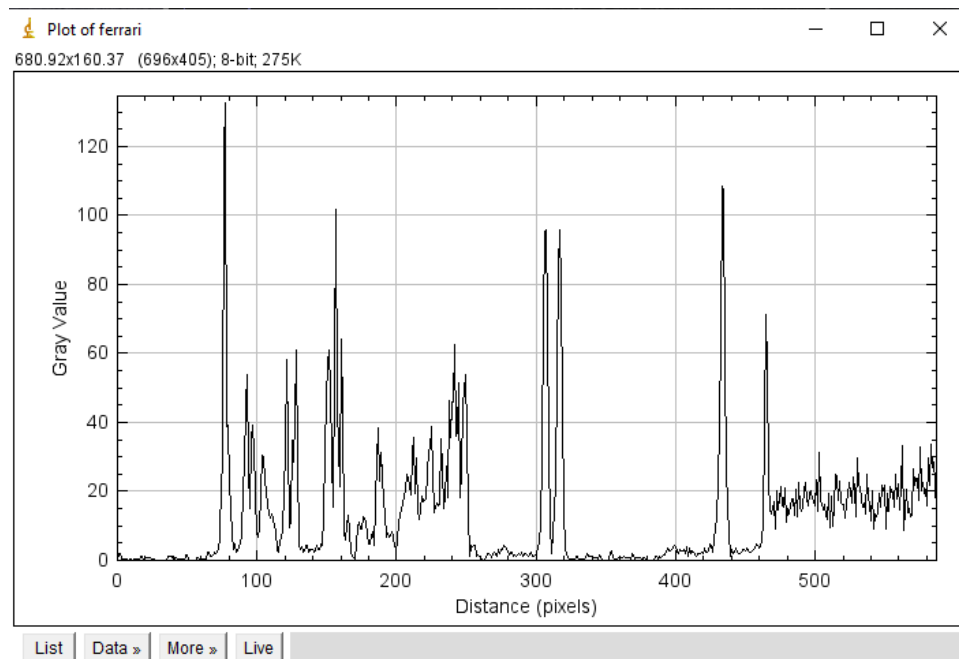
Avec la valeur absolue :



La découpe horizontale :



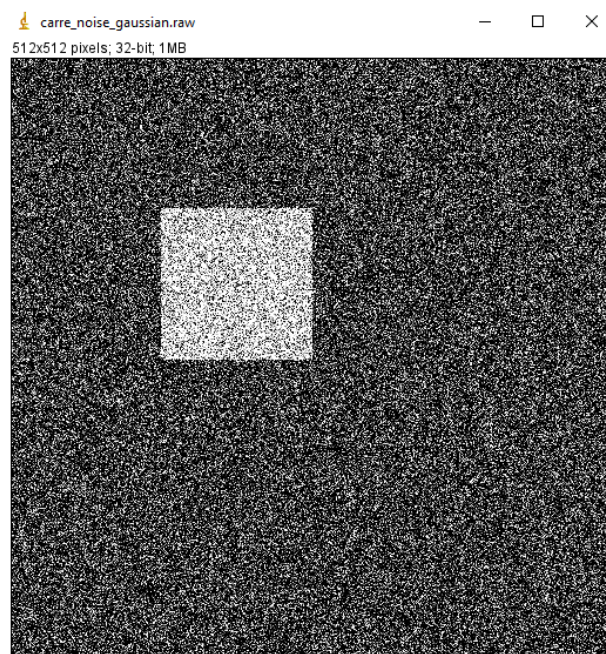
Le graphique (Gray value en fonction distance de pixel) :



On remarque, la résolution des valeurs de gris varie énormément cela est évident puisque la Ferrari possède plusieurs contours verticaux.

1.1.2 Images avec ajout de bruit gaussien :

Le bruit pollue souvent l'image et rend plus difficile la détection des contours, par exemple, l'image `carre_noise_gaussian.raw`.

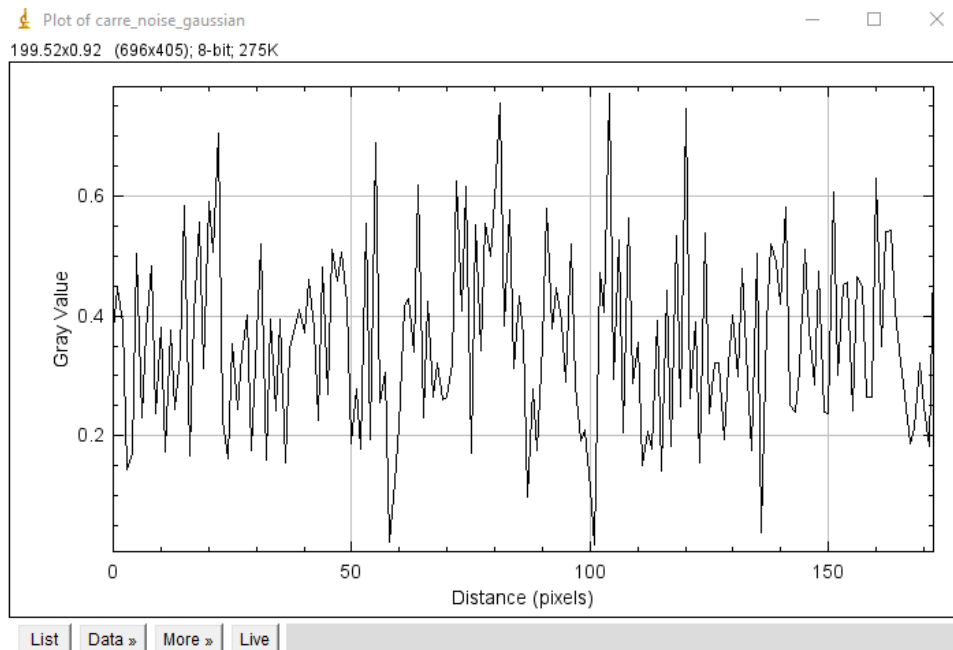


On remarque, plusieurs filaments gris qui viennent perturber l'image, donc, on peut imaginer la difficulté de contourner l'image.

Dérivée selon x avec la valeur absolue :

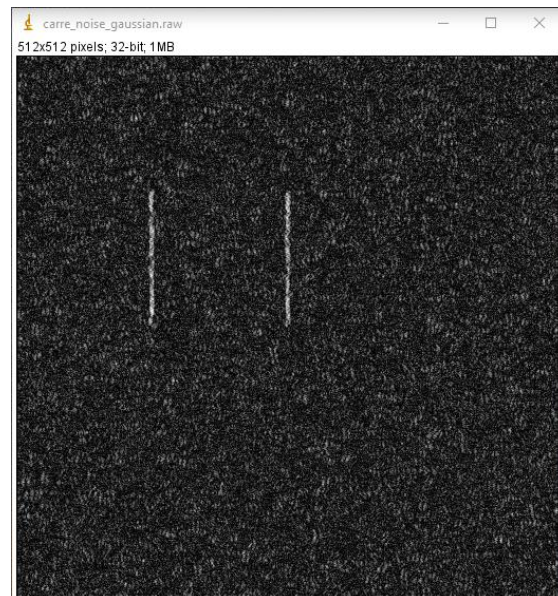


Le graphique (Gray value en fonction distance de pixel) sans filtre :

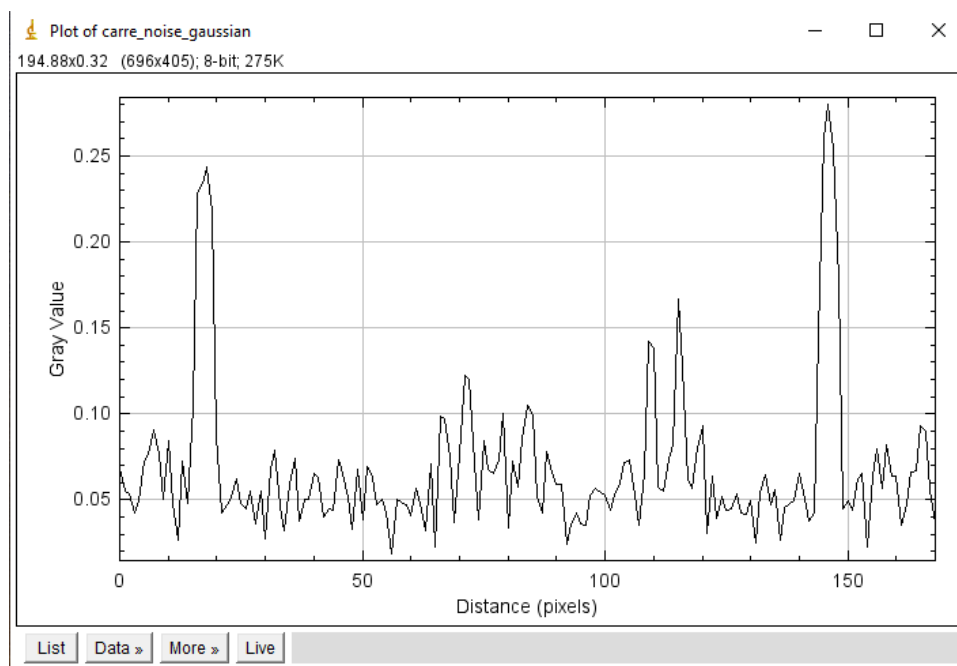


On remarque une légère présence de ligne verticales or cela ne convient pas, on remarque sur le graphique que la variation de la résolution des valeurs de gris est désordonnée. Donc, une technique simple de débruitage d'image pour la détection des contours en tant que prétraitement implique l'application d'un filtre de moyenne. Un pixel est remplacé par la moyenne des pixels environnants. Pour cela on utilise la commande : **Process->Filter->Mean**, puis appliquez l'opérateur de détection de contours verticaux, on a :

L'image filtre + $\text{abs}(Dx)$:



Le graphique (Gray value en fonction distance de pixel) avec filtre :



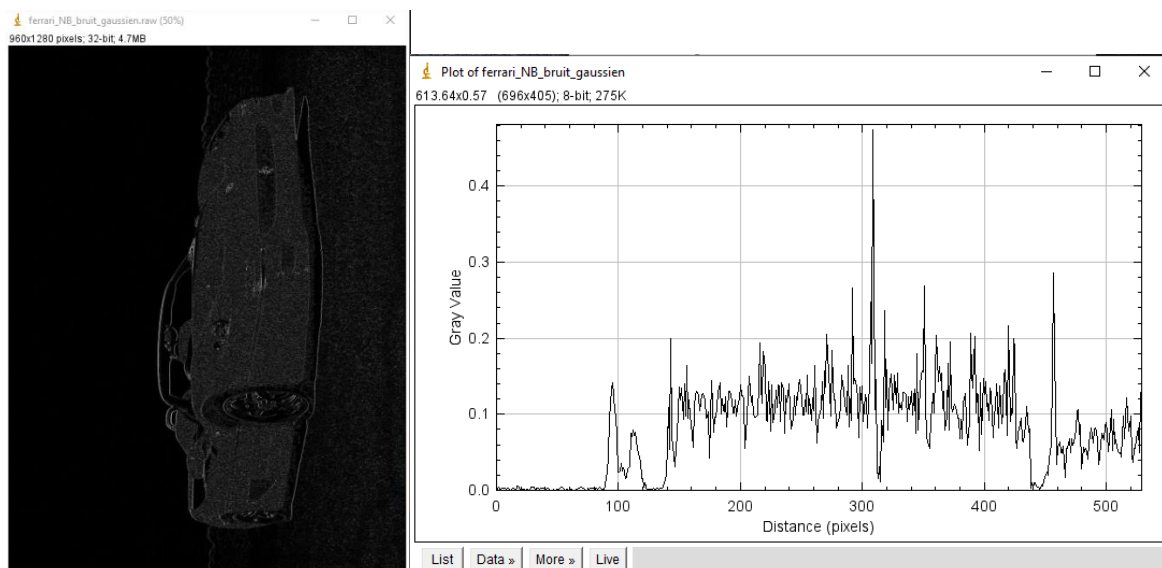
On remarque la variation de la résolution des valeurs de gris est plutôt correcte car on distinct bien deux pics importants de l'image.

J'ai réalisé les mêmes études que sur carre.raw sur l'image ferrari_NB.raw, j'ai obtenu les résultats suivants :



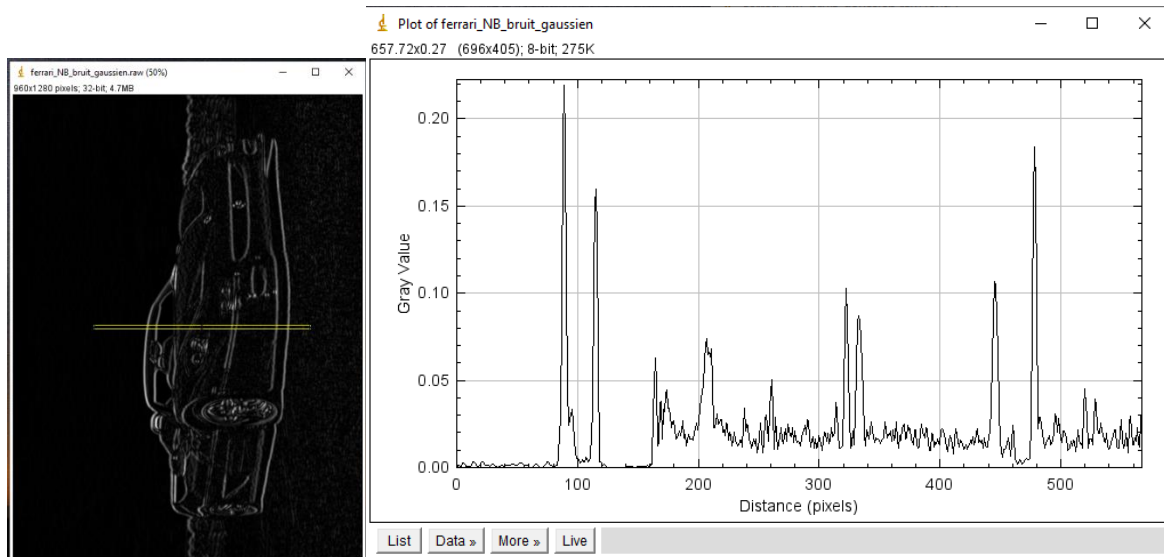
Dérivée selon x avec la valeur absolue :

Le graphique sans filtre :



L'image filtre + abs(Dx) :

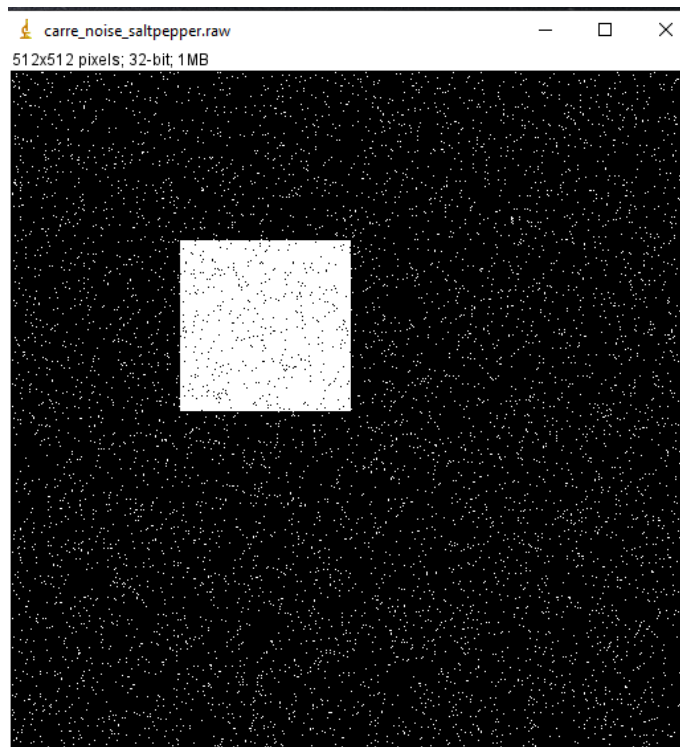
Le graphique avec filtre :



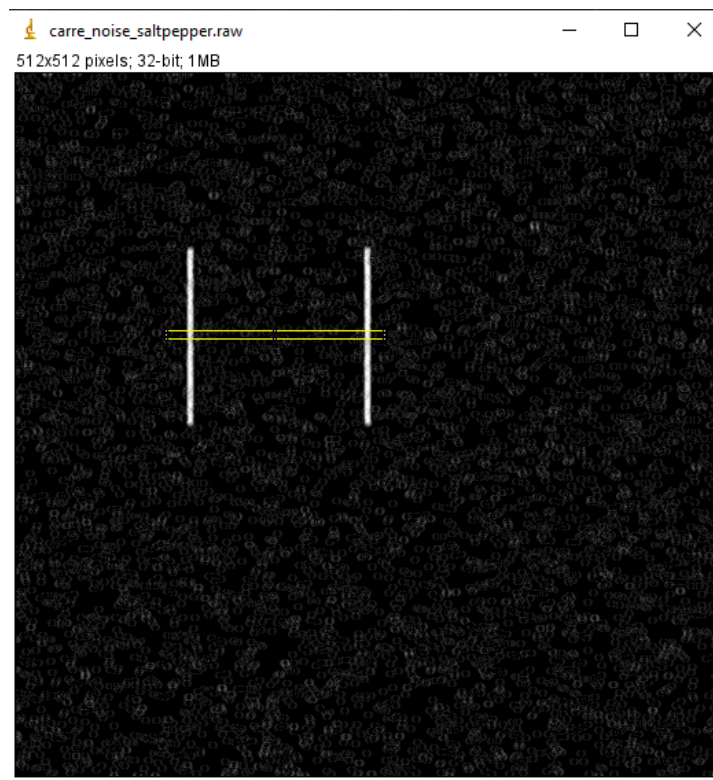
La même remarque pour le carrée...

1.1.3 Images avec ajout de bruit poivre et sel :

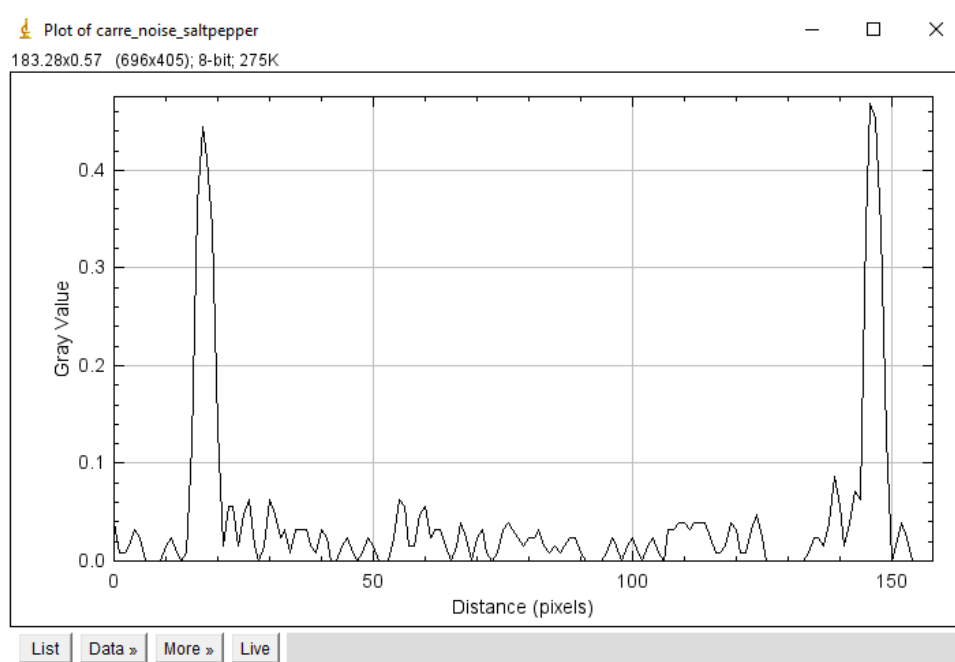
La nature du bruit peut différer du bruit gaussien, par exemple de bruit poivre et sel correspondant à un bruit impulsionnel.



Le filtre moyenneur puis l'opérateur de détection de contours verticaux :

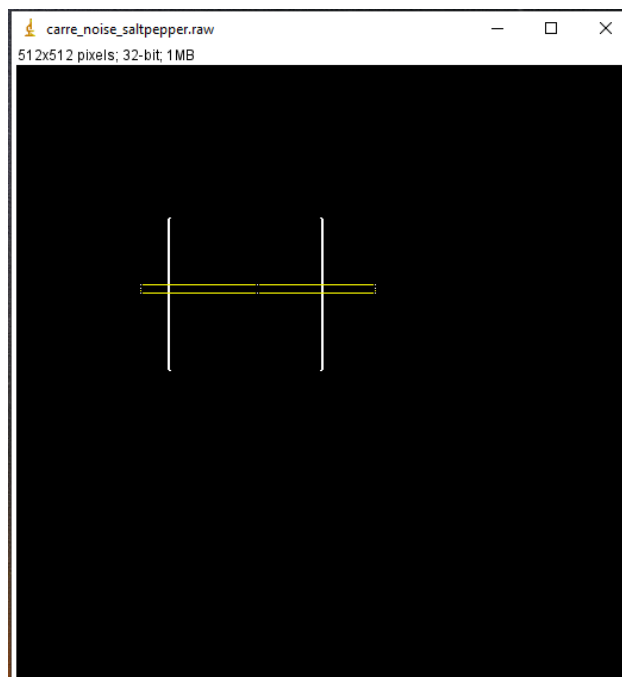


Graphique avec le filtre moyenneur puis l'opérateur de détection de contours verticaux :

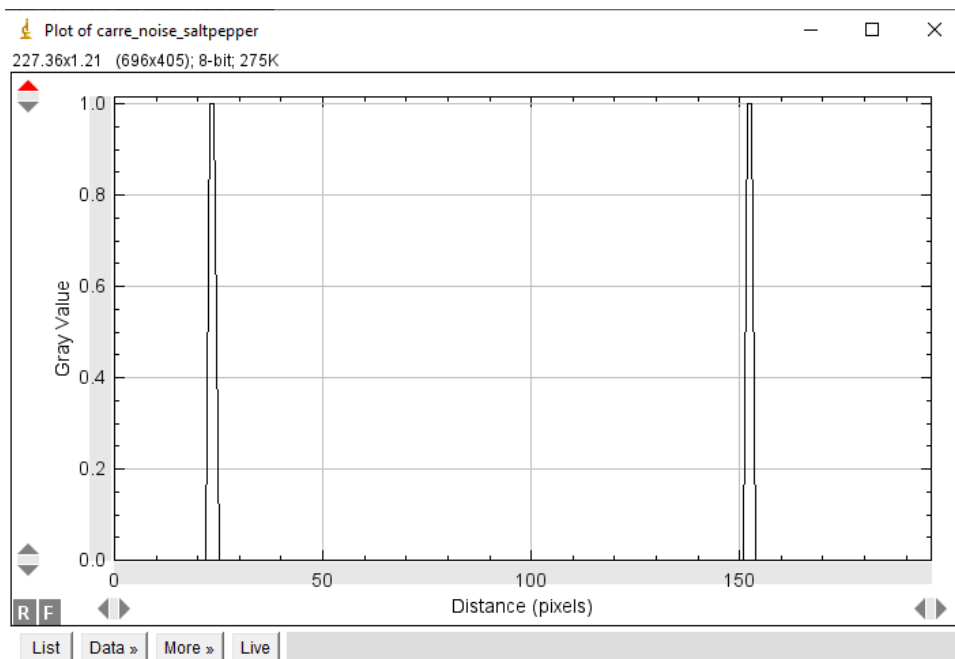


On remarque le filtre est plutôt efficace, Mais, l'utilisation d'un filtre médian est plus efficace, (Une technique plus adaptée que le filtre moyenneur comme débuteur est d'utiliser un filtre médian) on fait : **Process->Filter->Median**

Le filtre moyenneur puis l'opérateur de détection de contours verticaux :



Graphique avec le filtre moyenneur puis l'opérateur de détection de contours verticaux :

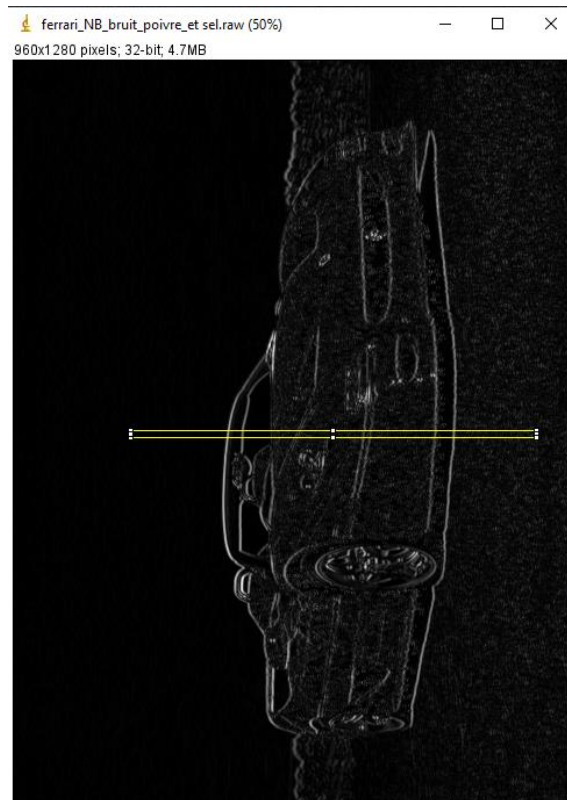


En effet l'efficacité de cette méthode se traduit par l'absence des bruits sur le graphique et l'image.

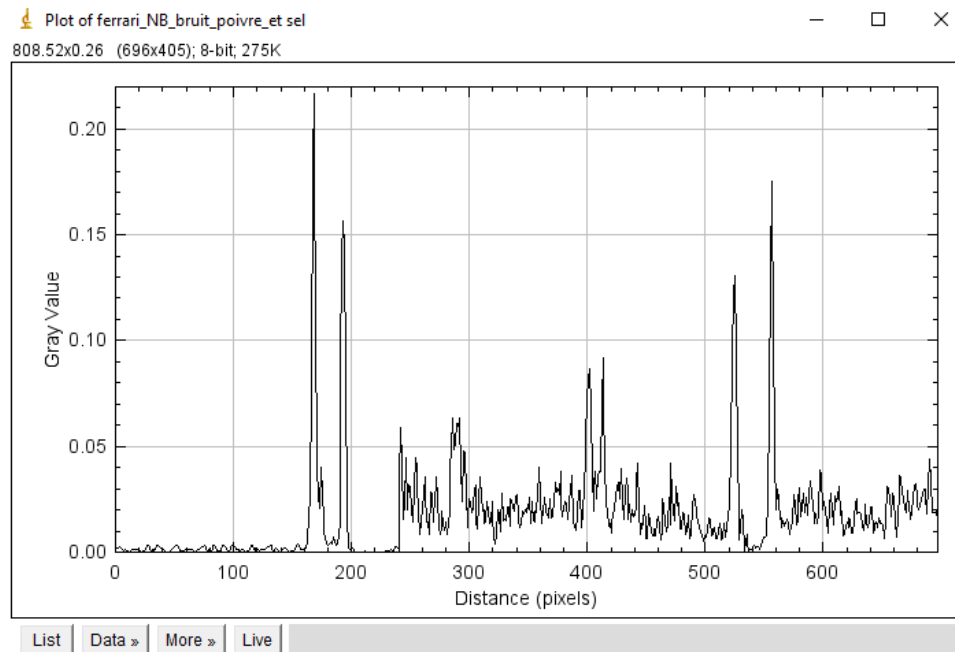
J'ai réalisé les mêmes études que sur carre.raw sur l'image ferrari_NB.raw, j'ai obtenu les résultats suivants :



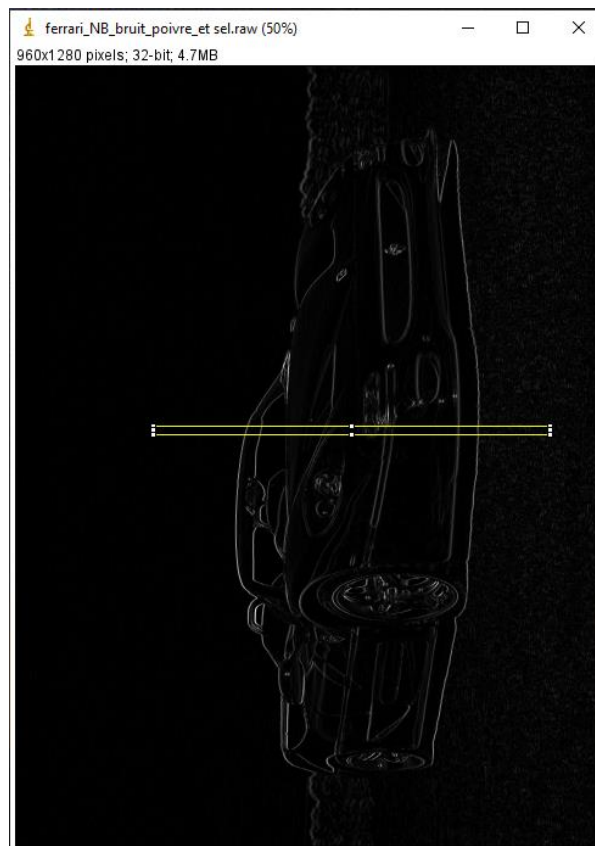
Le filtre moyenneur puis l'opérateur de détection de contours verticaux :



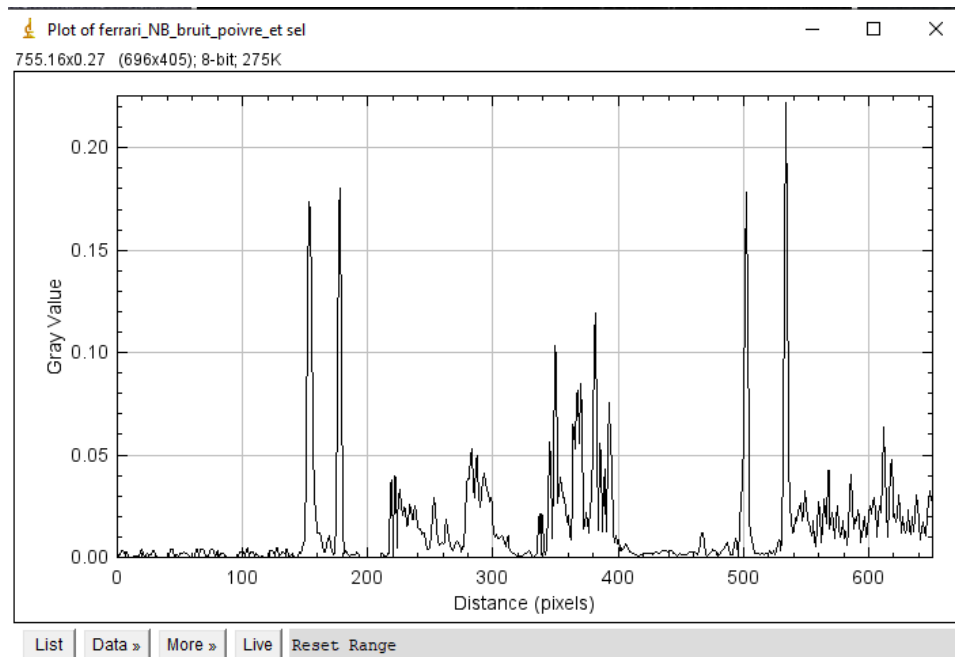
Graphique avec le filtre moyenneur puis l'opérateur de détection de contours verticaux :



Le filtre moyenneur puis l'opérateur de détection de contours verticaux :



Graphique avec le filtre moyennneur puis l'opérateur de détection de contours verticaux :



Effectivement avec la méthode de filtre médian on remarque une atténuation meilleure du bruit sur le graphique.

2 Estimation de performances sur cibles embarquées :

Dans les recherches précédentes sur l'implémentation d'algorithmes de traitement d'images, Nous voulons estimer le débit de calcul théorique de différentes architectures de calcul. Nous traitons un flux vidéo d'image de résolution 720p (720*1280 pixels).

2.1 On remarque que la complexité arithmétique d'un filtre 2D de taille Nnoyau est :

$$\text{On a : } \mathbf{Nop = N^2_{pixel} * N^2_{noyau} * 2}$$

Nop = nombre d'operation

Npixel = nombre de pixel (longueur * largeur)

Nnoyau = nombre de calcule/pixel

2 = car Accumulation de la mutiplication

d'où,

$$O(N^2 \text{pixel} * N^2 \text{noyau})$$

O = ordre grandeur de la suite qui va donner la vitesse de convergence ou divergence.

On a donc,

$$N_{\text{pixel}} = 720 * 1280.$$

$$N_{\text{noyau}} = 3, \text{ car 2D et c'est 3 par 3.}$$

$$D'où, Nop = (720 * 1280) * (3)^2 * (2) = 16588800$$

2.2 La complexité arithmétique du calcul du gradient avec un filtre de Sobel :

$$G = \text{sqrt}(Dx^2 + Dy^2)$$

Donc, le nombre operation du graduant,

$$NopG = N^2_{\text{pixel}} * (N^2_{\text{noyau}} * (\text{Accumulateur de la multiplication}) * (\text{nombre de dériver Dx Dy}) + \text{nombre opération})$$

$$\text{Ici, } N_{\text{pixel}} = 720 * 1280$$

$$N_{\text{noyau}} = 3$$

$$\text{Accumulateur de la multiplication} = 2$$

$$\text{Nombre de dériver Dx Dy} = 2$$

$$\text{Nombre opération : 2 pour les racines carrées + 1 pour l'addition + 1 pour le sqrt}$$

D'où,

$$NopG = (720 * 1280) * ((3)^2 * (2) * (2) + 4) = 36864000$$

2.3 Calcule de puissance de calcul en GFLOPS et les FPS :

$$\text{Freq} * \text{Coeur} * \text{Nb op}$$

$$\text{FPS} = (\text{Freq} * \text{Coeur} * \text{Nb op}) / Nop$$

Arch.	Carte	Modèle	Année (techn.)	Freq. (Ghz)	Coeurs	Nb op/coeur /cycle	Cons. (W)	Coût (\$)
CPU	Raspberry Pi 4	ARM A72	2015 (20 nm)	1.5	8	4	6,4 W	55
FPGA	Altera DE1Soc	Cyclone V	2011 (28 nm)	0.2	112	2	3	300
GPU	Jetson nano	Maxwell	2015 (20 nm)	0.92	128	2	10	99
GPU	Nvidia A100	Ampère	2020 (7 nm)	1.41	6912	2	400	12000

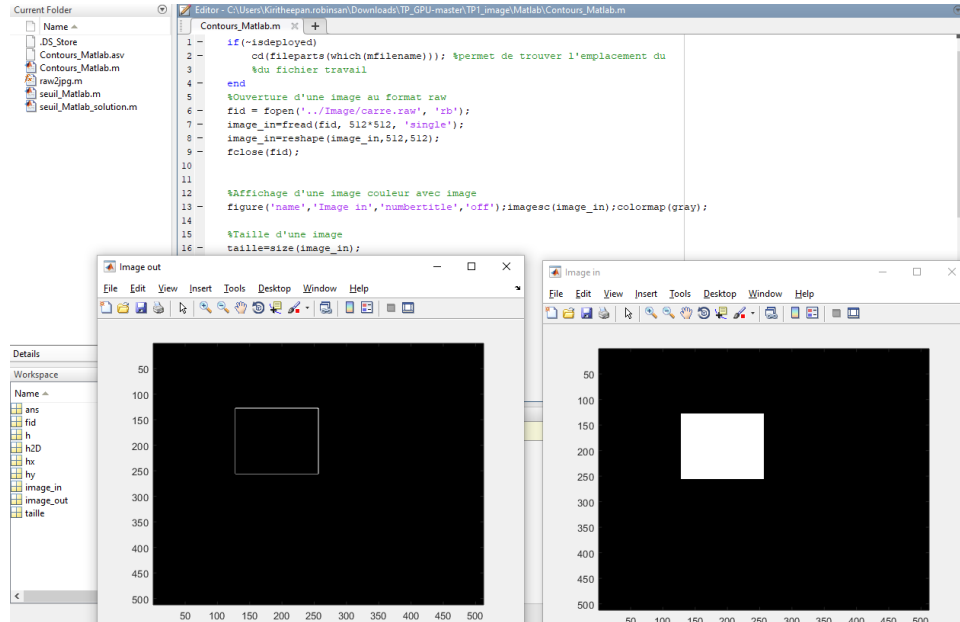
Par exemple, GPU du carte jetson nano modèle Maxwell on a :

$$\text{Puissance} = 0.92 * 10^9 * 128 * 2 = 2.3552 * 10^{11} \text{ W}$$

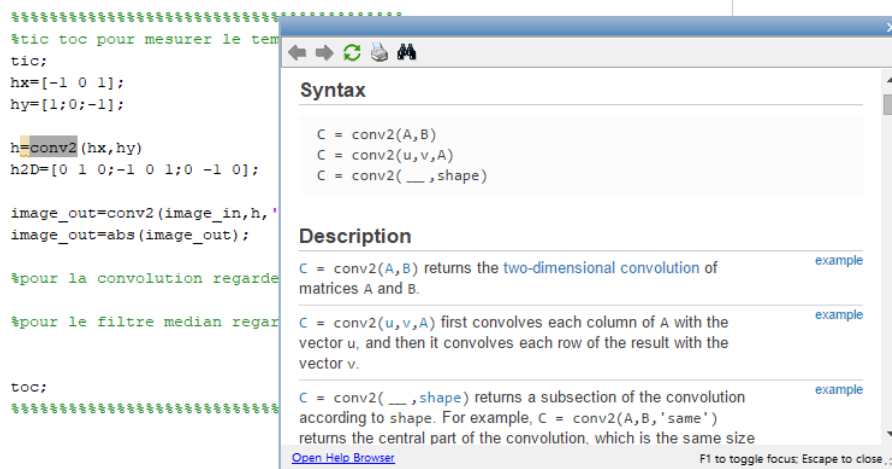
$$\text{FPS} = 235.52 / 16588800 = 14197.53086 \text{ FPS}$$

3 Matlab :

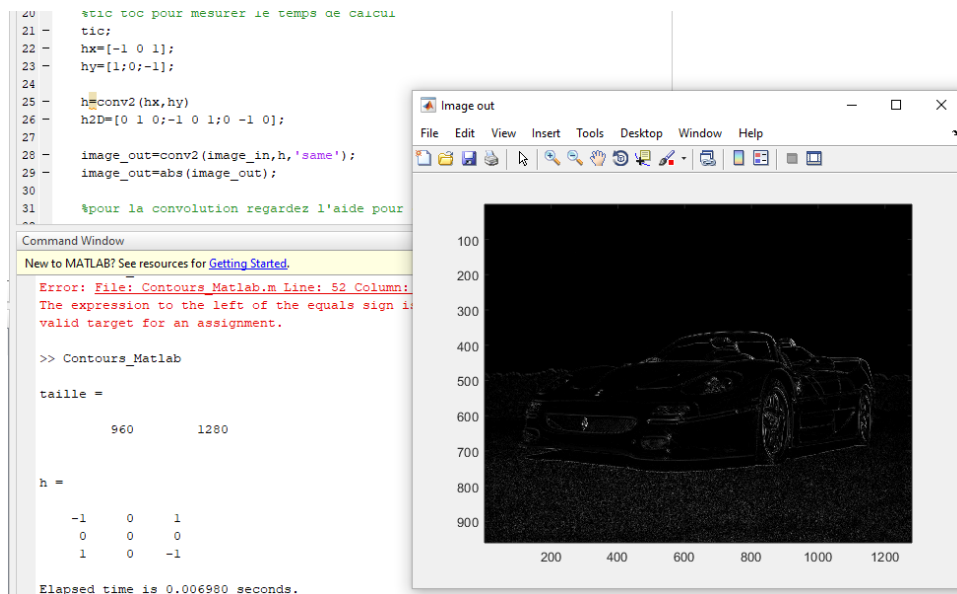
Effectuez sous Matlab, la détection de contours des images :



Du ligne 1 à 2 permet d'accéder plus facilement au dossier.

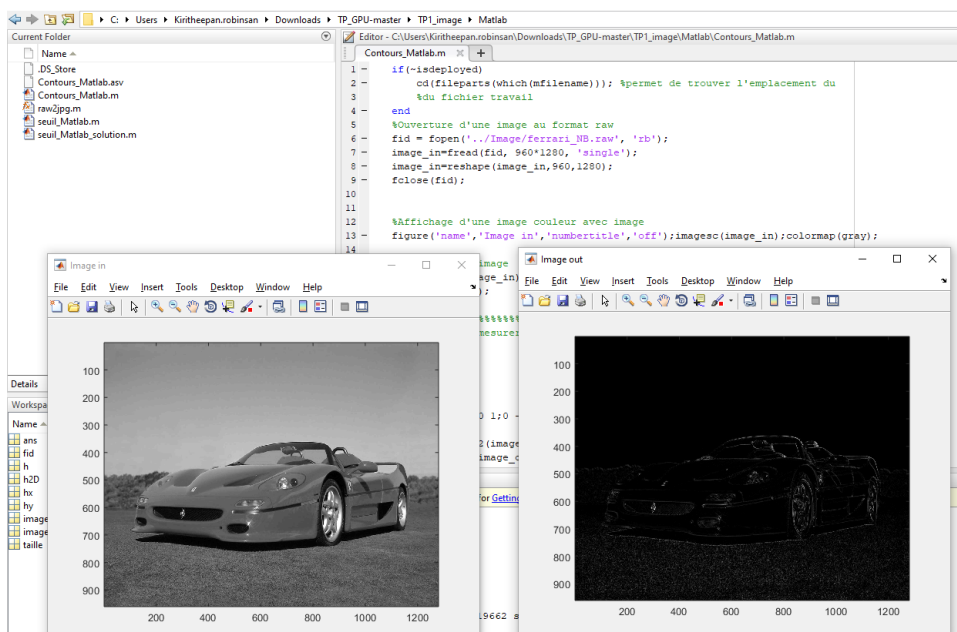


La méthode conv2 permet de réaliser des filtres.

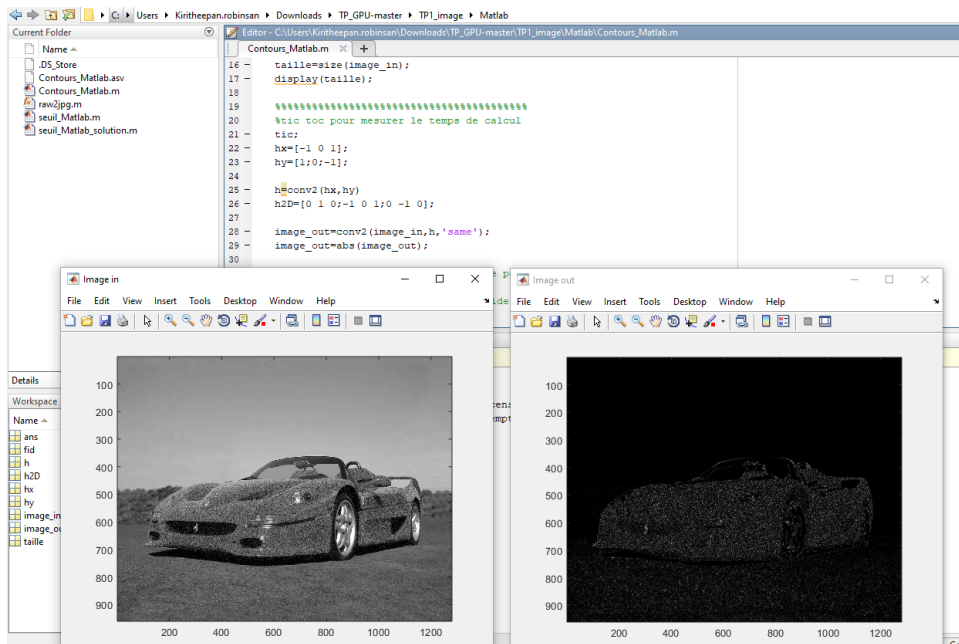


On peut voir sur le Command Windows la présence du matrice 3*3 et l'image à côté c'est donc le filtrage réaliser.

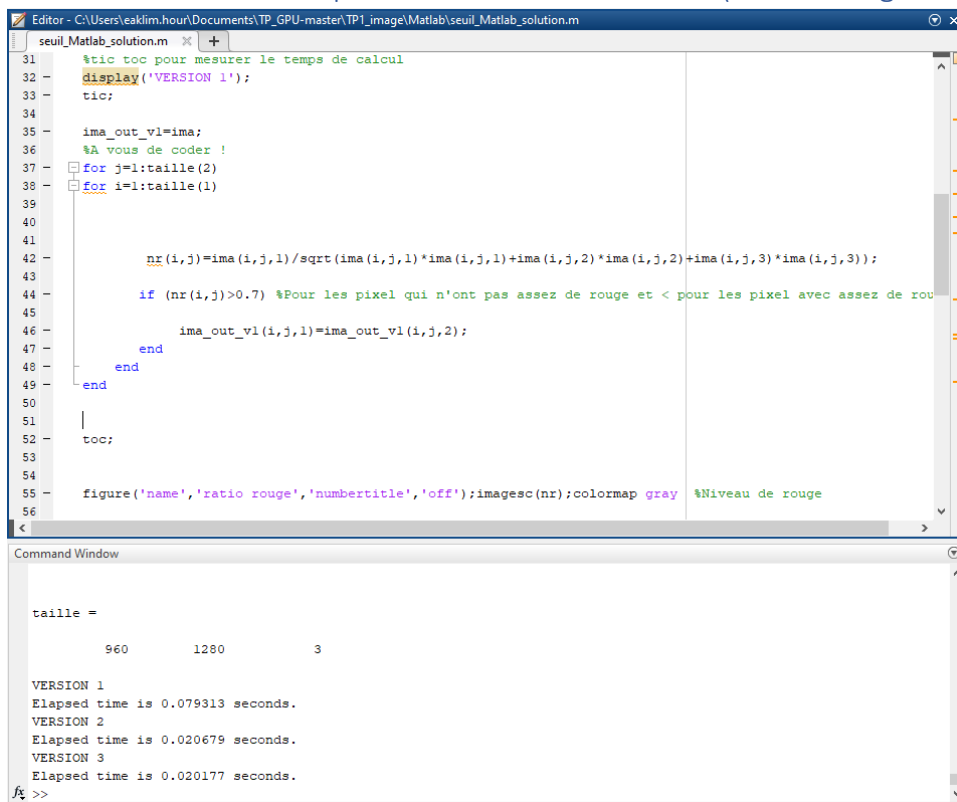
ferrari_NB.raw :



ferrari_NB_bruit_gaussien.raw :



3.1 Réduction du temps d'exécution sous Matlab (Ferrari rouge et Ferrari jaune) :



On avait 3 versions programme différent on a remarqué cela la façon qu'on l'écrit il était soit plus rapide ou plus longs. Pour avoir un ferra jaune faut mettre le rouge et le vert au même niveau :

Par exemple Out_rouge = In_rouge , Out_vert = In_vert et Out_bleu = In_bleu.

4 Traitement d'un flux vidéo (programmation C/C++) :

```
void seuillage_C(float image_out[][SIZE_J][SIZE_I], float image_in[][SIZE_J][SIZE_I])
{
    for (int j = 0; j < SIZE_J; j++){
        for (int i = 0; i < SIZE_I; i++){
            image_out[0][j][i]=image_in[0][j][i];
            image_out[2][j][i]=image_in[2][j][i];

            float nr =image_in[0][j][i]/sqrt(image_in[0][j][i]*image_in[0][j][i]+image_in[1][j][i]*image_in[1][j][i]+image_in[2][j][i]*image_in[2][j][i]);

            if (nr>0.7) {
                image_out[1][j][i]=image_in[0][j][i];
            }
            else
            image_out[1][j][i]=image_in[1][j][i];
        }
    }
}
```

Mon camarade et moi nous avons adapté le programme de Matlab sur C++.

5 Conclusion :

Dans le traitement image la notion de contourner une image par sa dérivée est importante on peut faire cela avec une dérivation horizontale ou verticale, or il est fort probable que les images contiennent du bruit, pour cela, il faudra utiliser des filtres selon le type de bruit. La rapidité du calcul est une notion importante faut connaître ce qu'est un langage compilé c'est-à-dire le programme est directement exécuté sur l'ordinateur, donc il sera en général plus rapide (C, C++) et un langage interprété c'est-à-dire un langage qui est lu ligne par ligne par un interpréteur, donc, c'est long (Matlab, python).